🦑 ScyllaDB GTM Hunter

Prompts & Agents Documentation

Home Assignment — GTM Engineer @ ScyllaDB

April 2026

Overview

This document captures all AI prompts, agent instructions, and tool-usage prompts used across two AI coding assistants — Claude.ai (claude.ai) and Claude Code (terminal) — to build the ScyllaDB GTM Hunter pipeline.

The pipeline automates competitor displacement outreach in 5 stages:

- Lead Discovery — Apollo API (mock) to find DataStax users
- Company & People Enrichment — technographic + contact data
- Qualification — 20-point scoring rubric per lead
- AI Copywriting — personalized LinkedIn invite + follow-up email per pain angle
- QA + Reporting — character-limit checks, SQLite persistence, HTML report

How AI Assistants Were Used

Tool	Role in this project
Claude.ai (chat)	Architecture decisions, apollo.py refactoring, pipeline design, prompt engineering for copywriting agent, documentation
Claude Code (terminal)	File scaffolding, iterative code editing, running tests, debugging, generating README

Section 1 — Claude.ai Chat Prompts

These are the key prompts given to Claude.ai (in the browser chat interface) that shaped the architecture, code, and pipeline design.

1.1 Initial Lead Identification Strategy

The first session explored how to identify DataStax users without scraping LinkedIn.

Prompt 1.1 — Lead Identification Approach

```
I'm applying for a GTM Engineer role at ScyllaDB. The home assignment asks me
to build an automated "hunter" workflow that finds users of DataStax (their
competitor) and sends them personalized outreach.

How would you identify relevant users on LinkedIn / the internet without
manual scraping? I want to see what APIs or data sources make sense for a
small-scale PoC.
```

Key outcome: Claude suggested using the Apollo.io API for technographic company search + people enrichment, and GitHub's free API (stargazers of DataStax repos) as a zero-cost alternative. The Apollo route was chosen for the final implementation.

1.2 Apollo.py — Switch from Python Import to JSON

The original apollo.py imported mock data from a Python module (src.data.apollo_mock_data). The prompt below asked Claude to refactor it to load from a standalone JSON file instead.

Prompt 1.2 — Refactor apollo.py Mock Data Loading

```
I would like that the apollo.py will use the json file and not trying to
import a py file.

[apollo.py and apollo_mock_data.json were uploaded]
```

Key outcome: Claude replaced the Python module import with a _load_mock_data() function that searches three candidate paths (same directory, src/data/, cwd) and loads the JSON at runtime. This made the connector portable with no module dependencies on internal paths.

1.3 Full Pipeline Architecture — Multi-Agent Design

After the Apollo connector was working, Claude was prompted to design and build the full 5-stage pipeline.

Prompt 1.3 — Build the Full GTM Hunter Pipeline

```
Now build the full GTM hunter pipeline that:
1. Uses apollo.py (mock mode) to find and enrich DataStax leads
2. Qualifies leads with a scoring rubric (seniority, company size, tech
signal, email status)
```

```
3. Uses Claude AI (via Anthropic API) to write a personalized LinkedIn invite
and follow-up email per lead — personalised by pain angle (latency / cost /
lock-in / scale)
4. QA checks the generated copy (character limits, forbidden words,
personalization check)
5. Saves everything to SQLite and generates an HTML report

Dry-run only — no real messages sent. Python. Single file is fine.
```

Key outcome: Claude produced pipeline.py with five agent classes — ResearcherAgent, QualifierAgent, CopywriterAgent, QAAgent, ReporterAgent — each with a clear single responsibility. The CopywriterAgent was designed to call the Anthropic API and fall back to templates if no key is provided.

1.4 Copywriter Agent — LLM System Prompt

The most critical prompt in the project: the system prompt passed to Claude (claude-sonnet-4-20250514) inside the CopywriterAgent to generate personalized outreach copy.

Prompt 1.4a — Claude.ai meta-prompt: design the copywriter system prompt

```
Write the system prompt I should use inside the CopywriterAgent when calling
the Anthropic API. It should instruct Claude to act as a B2B GTM specialist
writing outreach for ScyllaDB targeting DataStax users.

The prompt needs to:
- Receive structured lead context (name, title, company, technologies, pain
angle)
- Output a LinkedIn invite (< 300 chars) and a follow-up email (subject +
body)
- Be personalized to the specific pain angle: latency / cost / lock-in /
scalability
- Sound human, not like AI marketing copy
- Output as JSON so it's easy to parse
```

The resulting system prompt (embedded in pipeline.py) is shown below:

Prompt 1.4b — CopywriterAgent System Prompt (sent to Anthropic API)

```
You are a B2B GTM specialist at ScyllaDB — an open-source, high-performance
NoSQL
database that is a drop-in replacement for Apache Cassandra and DataStax.

Your task: write personalized outreach to engineers and technical leaders who
currently
use DataStax products, to make them aware of ScyllaDB as a superior
```

```
alternative.

CONTEXT YOU WILL RECEIVE:
- Lead name, title, company name and industry
- Technologies the company currently uses
- A pain_angle: one of latency | cost | lock-in | scalability
- A short pain_description explaining their specific challenge

WRITING RULES:
- Sound like a knowledgeable peer, not a sales robot
- Reference their specific tech stack and pain angle
- LinkedIn invite: MUST be under 300 characters including spaces. Be direct
and human.
- Email: professional but conversational. Subject line under 60 chars.
- Never say "I hope this message finds you well"
- Never use the phrase "game-changer"
- Always mention a concrete ScyllaDB advantage relevant to their pain angle
- Output ONLY valid JSON, no markdown, no preamble

OUTPUT FORMAT:
{
  "linkedin_invite": "...",
  "email_subject": "...",
  "email_body": "..."
}
```

1.5 HTML Report Design

Claude was asked to design the dark-theme HTML report that the ReporterAgent generates.

Prompt 1.5 — HTML Report Styling

```
The pipeline generates an HTML report of all leads and their generated copy.
Make it look professional — dark theme, teal accents matching ScyllaDB
branding, one card per lead showing: name, title, company, qualification
score, pain angle, LinkedIn invite, and email body. Should work when opened
directly in a browser (no server).
```

Key outcome: A self-contained dark-theme HTML template with inline CSS was generated and embedded in the ReporterAgent.

Section 2 — Claude Code (Terminal) Prompts

Claude Code was used for hands-on file operations, running scripts, debugging, and iterative edits. Below are the key prompts given in the terminal sessions.

2.1 Project Scaffolding

Claude Code Prompt 2.1 — Create project structure

```
Create a Python project called scylladb_hunter with this structure:
- pipeline.py (main entrypoint)
- apollo.py and apollo_mock_data.json (already provided, just place them)
- output/ directory for SQLite DB and HTML report
- README.md

Install dependencies: httpx anthropic
```

2.2 Debugging the Pipeline Run

Claude Code Prompt 2.2 — Debug pipeline execution errors

```
Run pipeline.py and fix any errors. The pipeline should:
- Print step-by-step progress logs
- End with a summary: leads loaded / qualified / QA passed / DB path / report
path
- Exit cleanly with code 0
```

2.3 Wiring the Anthropic API Key

Claude Code Prompt 2.3 — Add --api-key CLI argument

```
Add a --api-key argument to pipeline.py so the user can pass their Anthropic
key on the command line. It should override the ANTHROPIC_API_KEY environment
variable. If neither is set, fall back to mock copy templates (no crash).
```

2.4 README Generation

Claude Code Prompt 2.4 — Write the README

```
Write a README.md for this project covering:
- What it does and why (one paragraph)
- Pipeline diagram (ASCII)
- Quick start (dry-run and with API key)
- CLI options
- Output structure (report.html + SQLite schema)
- Design decisions (why mock Apollo, why deterministic QA, why pain-category
routing)
```

Section 3 — Claude Code Development Session

The following prompts were used during the full iterative development session in Claude Code. They cover the complete project lifecycle from initial review through final documentation.

3.1 Review Assignment & Identify Gaps

Prompt 3.1

```
Review the home_assignment_instructions.pdf — do you think the project is
complete? Do we need to improve anything?
```

What changed

No code changes — assessment only. Identified gaps: no follow-up sequence, no Docker, no pre-seeded demo state, output files scattered, README absent. Produced a prioritised list of improvements to work through.

Files affected: None (analysis only)

3.2 Move DB to Subfolder & Clean State

Prompt 3.2

```
Before we do that, please move the scylladb_hunter.db into a sub folder under
data called 'DB'. After doing so run main.py then truncate the DB and delete
the output folder and delete old reports and outreach for fresh runs.
```

What changed

- Moved DB to data/DB/scylladb_hunter.db
- Updated ReporterAgent.__init__ to create and use the data/DB/ path
- Ran pipeline to verify, then cleared DB and output for a clean slate

Files affected: agents/reporter.py, data/DB/

3.3 Full Verification Pass

Prompt 3.3

```
Double check everything, every single claim in what you produced and at the
end make a table of what you were able to verify.
```

What changed

- Systematically verified all README claims against source code and live runs
- Produced a verification table with pass/fail per claim
- Fixed several discrepancies: path references, argparse help text, directory labels in README

Files affected: README.md, minor fixes across agents

3.4 Pre-seed a Previous Run for Demo Realism

Prompt 3.4

```
I would like to insert a 'previous run' so when the user runs the job for the
first time he will see one record of skipped due to already reached.
```

What changed

- Pre-seeded data/DB/scylladb_hunter.db with Megan Chambers (VP of Engineering, VaultEdge Payments) — status: linkedin_sent, timestamp: 30 days ago
- Pre-seeded data/output/reports/report_20260321_120000.html showing her first-touch run
- On the reviewer's first python main.py, Megan appears in the Already Touched section

Files affected: data/DB/scylladb_hunter.db, data/output/reports/report_20260321_120000.html

3.5 Single Report Showing Megan Skipped

Prompt 3.5

```
I would like that the user will see only one report. In the report he will see
only one record of Megan. Then when he runs the job himself he will see Megan
was skipped.
```

What changed

- Ensured pre-seeded report shows only Megan (run_id 20260321_120000)
- Confirmed that subsequent runs add new leads and show Megan as Already Touched in a new timestamped report — original report preserved

Files affected: data/output/reports/report_20260321_120000.html

3.6 Keep Pre-seeded Report for Realism

Prompt 3.6

```
I would like you to keep the first report and outreach (Megan's Chamber run).
To make it more realistic.
```

What changed

- Kept report_20260321_120000.html as the permanent pre-seeded artefact
- Reset DB and output after test runs to restore the initial state cleanly

Files affected: data/DB/scylladb_hunter.db, data/output/ (cleaned)

3.7 Discuss LinkedIn vs Email Timing

Prompt 3.7

```
Is there a separation between the LinkedIn message and the follow-up email? In
'real life', will the lead receive both LinkedIn message and email at the same
time?
```

What changed

No code changes — design discussion. Identified that SenderAgent was dispatching both the invite and the follow-up email in the same step, which is not realistic. Proposed a two-step sequence mirroring real sales workflows.

Files affected: None (design only)

3.8 Build Two-Step Outreach Sequence

Prompt 3.8

```
Let's create an external step. After sending the LinkedIn invite, the sender
queries the DB and looks for records with linkedin_sent status. It checks if
the lead answered the LinkedIn message. If not, it checks the status timestamp
and if more than 5 days have passed, it sends the follow-up email.
```

What changed

- Created agents/follow_up.py (FollowUpAgent) with constants TRIGGER_STATUS and FOLLOW_UP_DELAY_DAYS
- Created follow_up.py as a standalone CLI entrypoint (Step 2)
- SenderAgent now only dispatches the LinkedIn invite; FollowUpAgent handles the follow-up email separately after 5+ days
- Updated agents/__init__.py to export FollowUpAgent

Files affected: agents/follow_up.py, follow_up.py, agents/sender.py, agents/__init__.py

3.9 Update Email Signature to Yotam Oppenheimer

Prompt 3.9

```
In the follow-up email change the name at the end from Alex Chen to Yotam
Oppenheimer. In the Megan initial report and for the following follow-up
emails.
```

What changed

- Updated all 9 email templates in copywriter.py (cold + second_touch) to sign with Yotam Oppenheimer, Solutions Engineer @ ScyllaDB
- Updated Megan's pre-seeded follow-up email body in the DB directly
- Updated the pre-seeded HTML report with the correct signature

Files affected: agents/copywriter.py, data/DB/scylladb_hunter.db, data/output/reports/report_20260321_120000.html

3.10 Design Follow-Up Integration into Report

Prompt 3.10

```
Ok I would like that in the generated report under the "Already Touched"
section, you will see an update regarding the follow-up email being sent, and
the email itself. explain what are you going to do before writing anything
```

What changed

No code changes — design only. Outlined the integration plan: FollowUpAgent runs before ReporterAgent inside main.py, updates Lead objects in memory, reporter renders the result in one combined report. Explained the three files that would need to change.

Files affected: None (design only)

3.11 Confirm Single-Report Architecture

Prompt 3.11

```
run the follow-up.py email before the reporter.py and then the reporter will
fetches data that the follow-up updated from the DB.
```

What changed

No code changes — confirmation only. Agreed on the approach: FollowUpAgent updates both DB and in-memory Lead objects so the reporter can render follow-up content without a second DB query.

Files affected: None (confirmation only)

3.12 Reset to Initial State

Prompt 3.12

```
reset to initial state (megan record only)
```

What changed

- Reset DB to Megan only (linkedin_sent, 30 days ago, follow-up email body populated)
- Deleted all test run reports and outreach JSON files
- Retained only report_20260321_120000.html

Files affected: data/DB/scylladb_hunter.db, data/output/ (cleaned)

3.13 Make Report Sections Collapsible

Prompt 3.13

```
I would like that the HTML report will be interactive meaning at start all the
sections will be closed and when pressing a section it will open the section
with the relevant information.
```

What changed

- Added CSS for collapsible sections (section-header, section-body, chevron)
- Added toggleSection() JavaScript function
- All four report sections start collapsed; clicking the header expands/collapses with a rotating chevron animation
- Refactored _build_html to use a reusable _section() helper

Files affected: agents/reporter.py

3.14 Improve Already Touched Status Display

Prompt 3.14

```
I would like to edit the Already Touched statuses. Instead of 'skipped' write
'LinkedIn message sent' (before sending a follow-up email). In the info
section write when the LinkedIn message was sent. Also timestamp when the
follow-up email was sent.
```

What changed

- Renamed the SKIPPED badge to 'LinkedIn Sent' in _message_type_badge
- Added _fmt_ts() helper to format ISO timestamps to human-readable strings
- For linkedin_sent leads: shows 'LinkedIn message sent on [date] — awaiting response before follow-up email'
- For email_sent leads: shows 'Follow-up email sent on [date] — LinkedIn invite was unanswered for 5+ days'
- Fixed CopywriterAgent._prior_contact_status to also fetch and restore

status_updated_at, status, follow-up email subject and body from DB for skipped leads

Files affected: agents/reporter.py, agents/copywriter.py

3.15 Fix Unknown Date Bug

Prompt 5.29

```
lead card shows unknow data, review the timestamp fetch from the db. below the
current lead crad info -
"LinkedIn message sent on unknown date — awaiting response before follow-up
email."
```

What changed

- Diagnosed root cause: CopywriterAgent marked leads as skipped but did not load status_updated_at from DB onto the in-memory Lead object
- Fixed _prior_contact_status to return status_updated_at and _write_copy to set it on the lead when skipping
- Also restored lead.status and copy fields from DB so email_sent leads render the FOLLOW-UP SENT badge and email content correctly on subsequent runs

Files affected: agents/copywriter.py

3.14 Design Multi-Variant CopywriterAgent

Prompt 3.14

```
I would like to create an improved CopywriterAgent. The CopywriterAgent will
generate multi templates for email and LinkedIn and then the QA chooses the
best option for each lead. create 3 templates for each lead and let the
QAagnet pick up the best option
```

What changed

- Added NUM_VARIANTS = 3 constant
- Replaced MOCK_COPY_TEMPLATES with MOCK_COPY_VARIANTS: 3 distinct angle variants per pain category (performance hook, technical detail, proof-point story) — 12 template sets total for cold outreach
- Added _generate_mock_variants() and _generate_live_variants() (3 Claude API calls with angle hints per lead in live mode)
- Added _apply_variants() to store all variants on the lead and default to V1
- Added Lead.copy_variants and Lead.qa_selected_variant fields
- Updated QAAgent: _score_variant() rubric, _variant_issues(), and _select_best_variant() — scores all variants, picks fewest-issues winner, logs per-variant breakdown
- Added 'QA: variant N/3' badge to the HTML report lead card

Files affected: agents/copywriter.py, agents/qa.py, models/lead.py, agents/reporter.py

3.15 Generate DB Inspector Script

Prompt 3.15

```
generate a py script under data/db that shows the DB tables meta data and also
show the data in each table
```

What changed

- Created data/DB/inspect_db.py — a standalone script that connects to scylladb_hunter.db and prints the schema and all rows for every table
- For each table: prints column names, types, and constraints (metadata), then prints all rows in a readable format
- Handles empty tables and missing DB gracefully with clear console output

Files affected: data/DB/inspect_db.py

Section 4 — Reused Prompt

The following prompts were used after every major change in the code.

4.1 Update README Agent Descriptions

Prompt 4.1

```
Update the README
```

4.2 Full Verification Pass

Prompt 4.2

```
Double check everything, every single claim in what you produced and at the
end make a table of what you were able to verify.
```